



Université Hassan II de Casablanca
Faculté des Sciences Ben M'Sik

Master Big Data et Data Science

Moteur de Recherche Sémantique
pour la Production Scientifique de la FSBM

Projet de Machine Learning et Natural Language Processing

Réalisé par : Anouar Mohammed

Encadré par : Elhabib Benlahmar

Année universitaire : 2025–2026

Casablanca, Maroc

Remerciements

Je tiens à exprimer ma gratitude à l'ensemble des personnes ayant contribué, directement ou indirectement, à la réalisation de ce projet.

Je remercie particulièrement mon professeur pour les orientations, les recommandations et les remarques qui ont permis de structurer ce travail et d'améliorer sa qualité technique.

Je remercie également la Faculté des Sciences Ben M'Sik de l'Université Hassan II de Casablanca pour le cadre académique dans lequel ce projet a été réalisé.

Enfin, je remercie toutes les personnes ayant contribué à la mise à disposition des ressources scientifiques utilisées dans le cadre de cette étude.

Résumé

Ce projet porte sur la conception et l'implémentation d'un moteur de recherche sémantique appliqué à la production scientifique de la Faculté des Sciences Ben M'Sik (FSBM).

L'objectif principal est de construire un pipeline complet permettant de collecter des informations scientifiques relatives aux enseignants-chercheurs, de nettoyer et structurer les données, de transformer les résumés scientifiques en représentations vectorielles et finalement de permettre une recherche basée sur la similarité sémantique.

La collecte des informations scientifiques est réalisée à partir de Google Scholar. Les données brutes sont ensuite soumises à une phase de nettoyage et de structuration permettant notamment de préserver les caractères accentués de la langue française, de détecter la langue des résumés, de supprimer les artefacts issus des pages web et de produire des fichiers aux formats JSON et Parquet.

Pour la représentation vectorielle, le projet utilise le modèle `zeroentropy/zembed-1-embedding`. Les documents scientifiques sont représentés par des vecteurs de dimension 2560. Ces vecteurs sont ensuite indexés dans une base de données vectorielle ChromaDB.

Enfin, une API REST développée avec FastAPI permet d'exposer le moteur de recherche, tandis qu'une interface Streamlit permet à l'utilisateur d'effectuer des recherches sémantiques de manière interactive.

Les expérimentations réalisées montrent la possibilité de retrouver des publications pertinentes à partir de requêtes formulées naturellement, même lorsque les termes employés dans la requête ne correspondent pas exactement aux termes présents dans les publications.

Mots-clés : NLP, recherche sémantique, embeddings, Zembed-1, ChromaDB, Google Scholar, FastAPI, Streamlit, Machine Learning.

Abstract

This project focuses on the design and implementation of a semantic search engine applied to the scientific production of the Faculty of Sciences Ben M'Sik (FSBM).

The main objective is to develop an end-to-end pipeline capable of collecting scientific information about researchers, cleaning and structuring the resulting data, transforming scientific abstracts into vector representations, and finally enabling semantic search based on vector similarity.

Scientific information is collected from Google Scholar and subsequently processed through a data cleaning pipeline. The preprocessing stage preserves French accented characters, detects the language of abstracts, removes web-related noise, and generates structured JSON and Parquet datasets.

For semantic representation, the project uses the **zeroentropy/zembed-1-embedding** model. Scientific documents are represented as 2560-dimensional vectors and stored in a ChromaDB vector database.

A FastAPI REST service exposes the semantic search functionality, while a Streamlit interface provides an interactive user experience.

The experiments demonstrate that semantic representations can retrieve relevant scientific publications from natural-language queries, including multilingual queries and queries whose wording does not exactly match the wording used in the indexed documents.

Keywords: NLP, Semantic Search, Embeddings, Zembed-1, ChromaDB, Google Scholar, FastAPI, Streamlit.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Introduction générale	9
1 Contexte et problématique	10
1.1 Présentation du contexte	10
1.2 Problématique	10
1.3 Objectifs	10
2 Architecture générale et méthodologie	12
2.1 Architecture du système	12
2.2 Organisation du projet	12
3 Collecte des données scientifiques	14
3.1 Source des données	14
3.2 Script de scraping	14
3.3 Gestion des erreurs	15
3.4 Checkpointing	15
3.5 Temporisation	15
4 Prétraitement et structuration des données	16
4.1 Problématique du nettoyage	16
4.2 Préservation des caractères français	16
4.3 Normalisation	16
4.4 Détection de langue	17
4.5 Dataset final	17
4.5.1 JSON	17
4.5.2 Parquet	17
4.6 Schéma des données	18
5 Représentation vectorielle avec Zembed-1	19
5.1 Principe des embeddings	19
5.2 Intégration de Zembed-1	19

5.3	Encodage des documents	19
5.4	Encodage des requêtes	20
5.5	Recherche par similarité	20
5.6	Vérification de la dimension	20
6	Indexation avec ChromaDB	22
6.1	Présentation	22
6.2	Indexation	22
6.3	Métadonnées	22
7	Conception du moteur de recherche sémantique	24
7.1	Principe général	24
7.2	Fonction de recherche	24
7.3	Requêtes multilingues	25
7.4	Exemple de requête	25
8	API FastAPI et interface Streamlit	26
8.1	API REST	26
8.2	Endpoint principal	26
8.3	Endpoint de santé	26
8.4	Interface Streamlit	27
8.5	Architecture finale	27
9	Évaluation et résultats	28
9.1	Objectifs de l'évaluation	28
9.2	Évaluation du scraping	28
9.3	Évaluation du prétraitement	28
9.4	Validation de Zembed-1	28
9.5	Tests de recherche	29
9.6	Recherche sémantique contre recherche lexicale	29
9.7	Visualisation de l'espace vectoriel	30
9.8	Illustrations des résultats	30
9.8.1	Interface de recherche	30
9.8.2	Résultats d'une requête	31
9.8.3	Projection PCA des embeddings	31
10	Reproductibilité et dépôt GitHub	32
10.1	Organisation du dépôt	32
10.2	Environnement Python	32
10.3	Exclusion des fichiers lourds	33

11 Limites et perspectives	34
11.1 Limites	34
11.1.1 Dépendance à Google Scholar	34
11.1.2 Disponibilité des résumés	34
11.1.3 Taille du modèle	34
11.1.4 Évaluation qualitative	34
11.2 Perspectives	34
Conclusion générale	36
A Commandes d'exécution	37
A.1 Création de l'environnement	37
A.2 Lancement de l'API	37
A.3 Lancement de Streamlit	37
A.4 Exemple de recherche	37
B Extrait de configuration Zembed-1	38
C Exemple de réponse de l'API	39

Table des figures

2.1	Architecture générale du moteur de recherche sémantique	12
8.1	Architecture d'exécution de la recherche sémantique	27
9.1	Interface Streamlit du moteur de recherche sémantique	30
9.2	Exemple de résultats retournés pour une requête en langage naturel	31
9.3	Projection PCA des embeddings des publications scientifiques	31

Liste des tableaux

4.1	Principaux attributs du dataset scientifique	18
9.1	Mécanismes de robustesse du scraping	28
9.2	Scénarios de test de recherche sémantique	29
10.1	Organisation des principaux fichiers	32

Introduction générale

La production scientifique d’une institution universitaire constitue une source importante d’information. Les publications scientifiques permettent d’identifier les domaines de recherche, les thématiques abordées par les chercheurs et les relations entre différentes disciplines.

Cependant, lorsque le volume de publications augmente, la recherche d’informations pertinentes devient progressivement plus difficile avec des méthodes classiques basées uniquement sur des mots-clés.

Une recherche lexicale traditionnelle cherche principalement des correspondances entre les termes de la requête et les termes présents dans les documents. Cette approche présente une limite importante : deux expressions différentes peuvent représenter une même idée ou un même concept.

Par exemple, une requête contenant l’expression « apprentissage automatique » peut être associée à des publications utilisant l’expression anglaise « machine learning ». Une recherche strictement lexicale peut ne pas exploiter efficacement cette relation sémantique.

La recherche sémantique vise à résoudre cette limitation en représentant les textes dans un espace vectoriel. Les textes ayant un sens proche sont alors représentés par des vecteurs proches selon une mesure de similarité.

Dans ce contexte, ce projet propose la conception d’un moteur de recherche sémantique appliqué aux publications scientifiques de la Faculté des Sciences Ben M’Sik.

Le travail est organisé autour de quatre grandes étapes :

1. collecte des informations scientifiques ;
2. nettoyage et structuration des données ;
3. génération des représentations vectorielles ;
4. recherche et restitution des publications pertinentes.

Le projet répond également aux trois livrables demandés :

- un dépôt GitHub contenant le code source ;
- un dataset scientifique final aux formats JSON et Parquet ;
- une démonstration commentée du fonctionnement du pipeline.

1. Contexte et problématique

1.1 Présentation du contexte

La Faculté des Sciences Ben M’Sik (FSBM), relevant de l’Université Hassan II de Casablanca, regroupe des enseignants-chercheurs travaillant dans différents domaines scientifiques.

Les publications associées à ces chercheurs sont disponibles à travers différentes sources scientifiques. Google Scholar constitue notamment une source permettant d’obtenir des informations sur les publications, les auteurs, les citations et les résumés lorsqu’ils sont disponibles.

L’exploitation automatique de ces informations permet de construire une représentation structurée de la production scientifique.

1.2 Problématique

La problématique principale peut être formulée comme suit :

Comment construire un système permettant de collecter, nettoyer, représenter et rechercher efficacement les publications scientifiques des chercheurs de la FSBM à partir de requêtes exprimées en langage naturel ?

Cette problématique peut être décomposée en plusieurs sous-problèmes :

- comment automatiser la collecte des profils et publications ?
- comment gérer les interruptions et les erreurs de collecte ?
- comment nettoyer correctement des textes scientifiques multilingues ?
- comment préserver les caractères accentués du français ?
- comment représenter les documents sous forme de vecteurs ?
- comment effectuer une recherche par proximité sémantique ?
- comment présenter les résultats à un utilisateur final ?

1.3 Objectifs

Les principaux objectifs du projet sont :

1. développer un scraper robuste ;
2. construire un dataset propre et structuré ;

3. utiliser le modèle Zembed-1 pour générer des embeddings ;
4. indexer les représentations dans ChromaDB ;
5. développer un moteur de recherche sémantique ;
6. fournir une API REST ;
7. proposer une interface graphique interactive ;
8. démontrer le fonctionnement du pipeline de bout en bout.

2. Architecture générale et méthodologie

2.1 Architecture du système

L'architecture globale du projet est organisée en plusieurs couches.

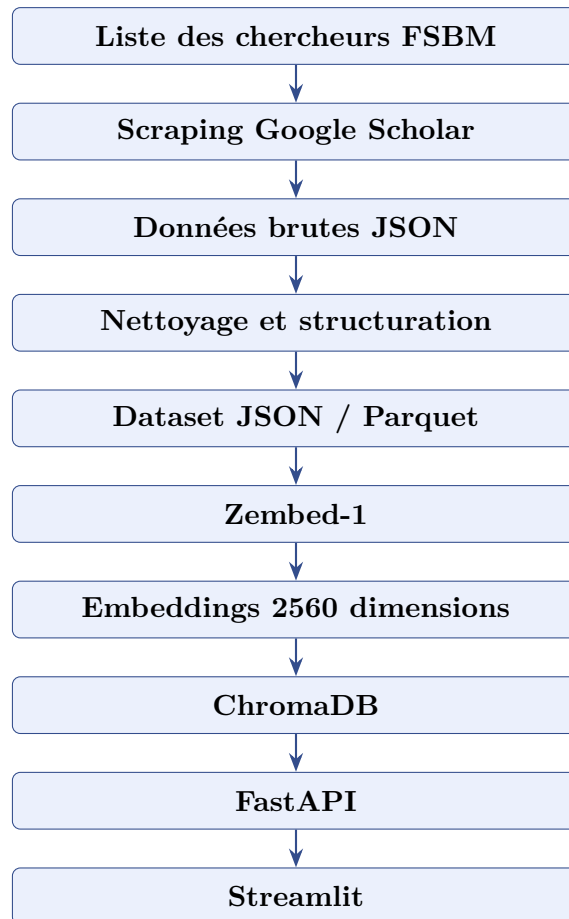


FIGURE 2.1 – Architecture générale du moteur de recherche sémantique

2.2 Organisation du projet

L'organisation des fichiers est la suivante :

```
1 projet_ml/  
2 |  
3 |-- app.py  
4 |-- requirements.txt  
5 |-- README.md  
6 |-- .gitignore  
7 |
```

```
8 |-- src/
9 |   |-- scraper.py
10 |   |-- cleaner.py
11 |   |-- generate_vectors_and_index.py
12 |   '-- api.py
13 |
14 |-- data/
15 |   |-- raw/
16 |   '-- processed/
17 |
18 '-- notebooks/
19     '-- semantic_search_demo.ipynb
```

Listing 2.1 – Structure principale du projet

Le fichier `scraper.py` est responsable de la collecte. Le fichier `cleaner.py` assure le nettoyage et la structuration. Le script `generate_vectors_and_index.py` génère les représentations vectorielles et réalise l'indexation. Le fichier `api.py` expose le moteur à travers FastAPI. Enfin, `app.py` fournit l'interface utilisateur Streamlit.

3. Collecte des données scientifiques

3.1 Source des données

La collecte repose sur Google Scholar et sur une liste initiale de chercheurs appartenant à la FSBM.

Pour chaque chercheur, les informations disponibles peuvent inclure :

- le nom complet ;
- l'identifiant du chercheur ;
- les publications ;
- le titre des publications ;
- les auteurs ;
- la date de publication ;
- le journal ou la source ;
- le nombre de citations ;
- le résumé lorsque celui-ci est disponible ;
- certaines métriques bibliométriques.

3.2 Script de scraping

Le script principal de collecte est :

```
src/scrapper.py
```

Le pipeline commence par le chargement du fichier :

```
data/raw/input_faculty_list.json
```

Les informations extraites sont ensuite sauvegardées dans :

```
data/raw/raw_scholar_data.json
```

3.3 Gestion des erreurs

Le scraping de Google Scholar constitue une partie sensible du projet en raison des mécanismes de limitation et de protection contre les requêtes automatisées.

Le système prévoit donc plusieurs mécanismes :

- gestion des exceptions ;
- sauvegarde progressive des résultats ;
- reprise après interruption ;
- temporisation entre les requêtes ;
- limitation de la fréquence des requêtes ;
- gestion des profils non trouvés ;
- gestion des publications incomplètes.

3.4 Checkpointing

Le principe de checkpointing permet d'éviter de perdre l'ensemble des résultats lorsqu'une exécution est interrompue.

Après l'acquisition des informations, celles-ci sont sauvegardées dans le fichier de données brutes.

Ainsi, une nouvelle exécution peut exploiter les informations déjà collectées au lieu de recommencer systématiquement toute la collecte.

Cette stratégie améliore la robustesse globale du pipeline.

3.5 Temporisation

Une temporisation entre les requêtes est utilisée afin de réduire la fréquence des requêtes automatiques.

Le principe du *jitter*, c'est-à-dire l'utilisation de délais variables, permet d'éviter un comportement strictement périodique des requêtes.

Cette approche constitue un mécanisme de réduction du risque de blocage, sans prétendre supprimer complètement les limitations imposées par le service source.

4. Prétraitement et structuration des données

4.1 Problématique du nettoyage

Les données provenant du Web peuvent contenir différents types de bruit :

- balises HTML ;
- caractères spéciaux ;
- espaces multiples ;
- métadonnées éditoriales ;
- éléments de copyright ;
- textes incomplets ;
- caractères mal encodés.

Une étape de nettoyage est donc nécessaire avant la génération des embeddings.

4.2 Préservation des caractères français

Un problème particulier concerne les caractères accentués.

Une suppression trop agressive des caractères non ASCII pourrait transformer un texte comme :

détection du cancer

en une forme incorrecte.

Le pipeline utilise donc une stratégie compatible avec les caractères Unicode du français afin de préserver les lettres accentuées.

Cette étape est particulièrement importante pour une institution francophone telle que la FSBM.

4.3 Normalisation

Les opérations de nettoyage comprennent notamment :

1. décodage des entités HTML ;
2. suppression des balises HTML ;

3. normalisation des espaces ;
4. nettoyage des caractères parasites ;
5. suppression de certains artefacts éditoriaux ;
6. validation de la longueur des résumés ;
7. détection de la langue ;
8. dédoublonnage.

4.4 Détection de langue

Le dataset peut contenir des publications en français et en anglais.

Une colonne dédiée permet d'indiquer la langue détectée :

`fr` ou `en`

Cette information peut ensuite être utilisée pour analyser la distribution linguistique du corpus.

4.5 Dataset final

Le dataset nettoyé est exporté sous deux formats.

4.5.1 JSON

Le format JSON conserve une structure adaptée à la manipulation des données scientifiques et des métadonnées.

4.5.2 Parquet

Le format Parquet fournit une représentation tabulaire efficace, particulièrement adaptée au traitement avec Pandas et aux analyses de données.

Le fichier principal est :

`data/processed/processed_abstracts.parquet`

Le fichier JSON correspondant est :

`data/processed/processed_abstracts.json`

4.6 Schéma des données

Les informations associées aux publications peuvent être organisées autour des champs suivants :

TABLE 4.1 – Principaux attributs du dataset scientifique

Attribut	Description
nom_complet	Nom du chercheur
chercheur_id	Identifiant du chercheur
titre	Titre de la publication
auteurs_str	Liste des auteurs
date_publication	Date ou année de publication
journal	Source ou journal scientifique
citations	Nombre de citations
abstract_clean	Résumé nettoyé
lang	Langue du résumé
word_count	Nombre de mots

5. Représentation vectorielle avec Zembed-1

5.1 Principe des embeddings

Un embedding est une représentation numérique d'un texte sous la forme d'un vecteur dans un espace de dimension élevée.

On peut représenter un document d par :

$$\mathbf{v}_d \in \mathbb{R}^{2560}$$

Dans le cadre de ce projet, le modèle utilisé est :

zeroentropy/zembed-1-embedding

5.2 Intégration de Zembed-1

Le modèle est chargé avec Sentence Transformers.

```
1 from sentence_transformers import SentenceTransformer
2
3 MODEL_NAME = "zeroentropy/zembed-1-embedding"
4
5 model = SentenceTransformer(
6     MODEL_NAME,
7     trust_remote_code=True
8 )
```

Listing 5.1 – Chargement du modèle Zembed-1

La dimension de sortie obtenue dans le projet est :

$$d = 2560$$

Cette dimension doit être identique entre les documents indexés et les requêtes utilisateur.

5.3 Encodage des documents

Lors de l'indexation, chaque résumé scientifique est transformé en vecteur.

On peut représenter cette opération par :

$$f(d) = \mathbf{v}_d$$

où f représente le modèle Zembed-1 et $\mathbf{v}_d$ le vecteur associé au document.

Les vecteurs obtenus sont ensuite stockés dans ChromaDB.

5.4 Encodage des requêtes

Lorsqu'un utilisateur saisit une requête q , celle-ci est également transformée en vecteur :

$$f(q) = \mathbf{v}_q$$

Le système cherche alors les documents dont les vecteurs sont les plus proches du vecteur de la requête.

5.5 Recherche par similarité

La similarité entre deux vecteurs peut être exprimée à partir de la similarité cosinus :

$$\text{cosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Lorsque les vecteurs sont normalisés, cette mesure permet de comparer leur proximité directionnelle.

Dans le contexte de ChromaDB, la distance cosinus peut être représentée par :

$$D(\mathbf{u}, \mathbf{v}) = 1 - \text{cosine}(\mathbf{u}, \mathbf{v})$$

Ainsi, une distance plus faible correspond à des représentations plus proches.

5.6 Vérification de la dimension

Une vérification importante est réalisée lors du démarrage de l'API.

Le système compare :

$$d_{\text{modèle}}$$

et :

$$d_{\text{ChromaDB}}$$

Le système doit satisfaire :

$$d_{\text{modèle}} = d_{\text{ChromaDB}}$$

Dans le projet :

$$2560 = 2560$$

Cette vérification permet d'éviter une erreur fréquente lors de l'utilisation de modèles d'embedding différents.

6. Indexation avec ChromaDB

6.1 Présentation

ChromaDB est utilisé comme base de données vectorielle locale.

La collection utilisée dans le projet est :

```
research_papers_db
```

La base est persistée dans :

```
data/vector_store/
```

6.2 Indexation

Le processus d'indexation suit les étapes suivantes :

1. chargement du dataset nettoyé ;
2. extraction des résumés ;
3. génération des embeddings ;
4. association des métadonnées ;
5. insertion des vecteurs dans ChromaDB.

Chaque document est donc associé à :

- son texte ;
- son embedding ;
- ses métadonnées.

6.3 Métadonnées

Les métadonnées permettent d'enrichir les résultats de recherche.

Par exemple :

- titre ;
- nom du chercheur ;
- identifiant du chercheur ;
- année ;

- journal ;
- nombre de citations ;
- langue.

Ainsi, le moteur ne renvoie pas seulement une distance numérique, mais également les informations permettant d'identifier et d'interpréter la publication.

7. Conception du moteur de recherche sémantique

7.1 Principe général

Le moteur reçoit une requête en langage naturel.

Par exemple :

Diagnostic du cancer du sein par apprentissage automatique

La requête est convertie en embedding.

$$q \rightarrow f(q)$$

Puis ChromaDB recherche les documents les plus proches :

$$f(q) \rightarrow \{d_1, d_2, \dots, d_k\}$$

Les résultats sont finalement affichés avec leurs métadonnées.

7.2 Fonction de recherche

La fonction principale peut être résumée comme suit :

```
1 def search_fsbm_papers(query_text, top_k=4):
2
3     query_vector = model.encode(
4         query_text,
5         normalize_embeddings=True
6     ).tolist()
7
8     results = collection.query(
9         query_embeddings=[query_vector],
10        n_results=top_k
11    )
12
13    return results
```

Listing 7.1 – Principe de la recherche sémantique

7.3 Requêtes multilingues

La démonstration comprend plusieurs types de requêtes :

- intelligence artificielle et diagnostic médical ;
- sécurité des réseaux IoT ;
- traitement automatique du langage naturel ;
- mathématiques appliquées.

Les requêtes sont formulées en français et en anglais afin d'examiner le comportement du moteur sur un corpus multilingue.

7.4 Exemple de requête

Une requête de test est :

Diagnostic du cancer du sein par apprentissage automatique

Le système encode cette requête puis recherche les publications les plus proches dans l'espace vectoriel.

Les résultats retournés contiennent notamment :

- le titre ;
- le chercheur ;
- l'année ;
- le journal ;
- les citations ;
- la distance sémantique ;
- un extrait du résumé.

8. API FastAPI et interface Streamlit

8.1 API REST

Afin de séparer la logique de recherche de l'interface utilisateur, une API REST a été développée avec FastAPI.

L'application est définie dans :

```
src/api.py
```

8.2 Endpoint principal

L'endpoint de recherche est :

```
GET /search
```

Il accepte notamment :

- une requête textuelle ;
- le nombre maximal de résultats.

Exemple conceptuel :

```
1 GET /search?query=machine%20learning&top_k=5
```

Listing 8.1 – Exemple d'appel de l'API

8.3 Endpoint de santé

Un endpoint supplémentaire est disponible :

```
GET /health
```

Il permet de vérifier :

- l'état du service ;
- le modèle utilisé ;
- la dimension des embeddings ;
- la collection ChromaDB ;
- le nombre de documents indexés.

8.4 Interface Streamlit

Une interface interactive est développée dans :

`app.py`

L'utilisateur peut saisir une requête et obtenir directement les publications les plus pertinentes.

L'interface présente notamment :

- la requête ;
- le nombre de résultats ;
- les titres des publications ;
- les chercheurs ;
- les résumés ;
- les années ;
- les citations ;
- les distances sémantiques.

8.5 Architecture finale

L'architecture d'exécution peut être représentée ainsi :



FIGURE 8.1 – Architecture d'exécution de la recherche sémantique

9. Évaluation et résultats

9.1 Objectifs de l'évaluation

L'évaluation du projet est basée sur les critères définis dans le cahier des charges :

1. robustesse du scraping ;
2. qualité du prétraitement ;
3. intégration correcte de Zembed-1 ;
4. pertinence de la recherche sémantique.

9.2 Évaluation du scraping

La robustesse du scraping est assurée par plusieurs mécanismes :

TABLE 9.1 – Mécanismes de robustesse du scraping

Mécanisme	Objectif
Checkpointing	Éviter de perdre les données collectées en cas d'interruption.
Temporisation	Réduire la fréquence des requêtes automatisées.
Jitter	Introduire des délais variables entre les requêtes.
Gestion des exceptions	Éviter l'arrêt brutal du pipeline.
Validation des données	Éviter d'intégrer des profils ou publications incomplets.

9.3 Évaluation du prétraitement

Le prétraitement répond à plusieurs contraintes importantes.

Premièrement, les caractères accentués français sont préservés. Deuxièmement, les résumés sont nettoyés des éléments parasites issus du Web. Troisièmement, les données sont exportées sous des formats structurés adaptés à leur exploitation ultérieure.

Le double export JSON/Parquet facilite également l'utilisation du dataset par différents outils.

9.4 Validation de Zembed-1

L'intégration de Zembed-1 est vérifiée par plusieurs contrôles :

- chargement réussi du modèle ;

- récupération de la dimension des embeddings ;
- vérification de la dimension ChromaDB ;
- comparaison des deux dimensions ;
- génération des embeddings de requêtes ;
- interrogation de la collection vectorielle.

Dans l'implémentation finale :

$$d_{\text{Zembed-1}} = 2560$$

et la collection ChromaDB utilise également des vecteurs de :

$$d_{\text{ChromaDB}} = 2560.$$

La cohérence dimensionnelle est donc respectée.

9.5 Tests de recherche

Plusieurs requêtes thématiques sont utilisées dans le notebook de démonstration.

TABLE 9.2 – Scénarios de test de recherche sémantique

Test	Thématique
Test 1	Diagnostic médical et apprentissage automatique
Test 2	Sécurité IoT et réseaux de capteurs sans fil
Test 3	NLP et chatbots intelligents
Test 4	Mathématiques appliquées et régularisation

Ces scénarios permettent de vérifier que le moteur peut être interrogé sur plusieurs domaines scientifiques.

9.6 Recherche sémantique contre recherche lexicale

Une comparaison entre une recherche lexicale et une recherche sémantique est également réalisée.

La recherche lexicale repose sur la présence exacte d'un mot ou d'une expression dans le titre ou le résumé.

La recherche sémantique repose quant à elle sur la proximité entre les représentations vectorielles.

On peut représenter les deux approches ainsi :

Recherche lexicale → Correspondance de termes

alors que :

Recherche sémantique → Proximité dans l'espace vectoriel

Cette différence permet à la recherche sémantique de prendre en compte des formulations différentes mais conceptuellement proches.

9.7 Visualisation de l'espace vectoriel

Le projet comprend également une visualisation PCA.

Les embeddings de dimension 2560 sont projetés dans un espace bidimensionnel :

$$\mathbb{R}^{2560} \rightarrow \mathbb{R}^2$$

La méthode utilisée est l'analyse en composantes principales (PCA).

Cette visualisation permet d'explorer la distribution des publications dans l'espace sémantique et d'observer d'éventuels regroupements thématiques (voir la figure 9.3).

9.8 Illustrations des résultats

Cette section regroupe les trois figures illustrant le fonctionnement du moteur de recherche sémantique : l'interface utilisateur, un exemple de résultats de recherche et la visualisation de l'espace vectoriel.

9.8.1 Interface de recherche

La figure 9.1 présente l'interface Streamlit permettant de saisir une requête en langage naturel et de consulter les publications retournées.

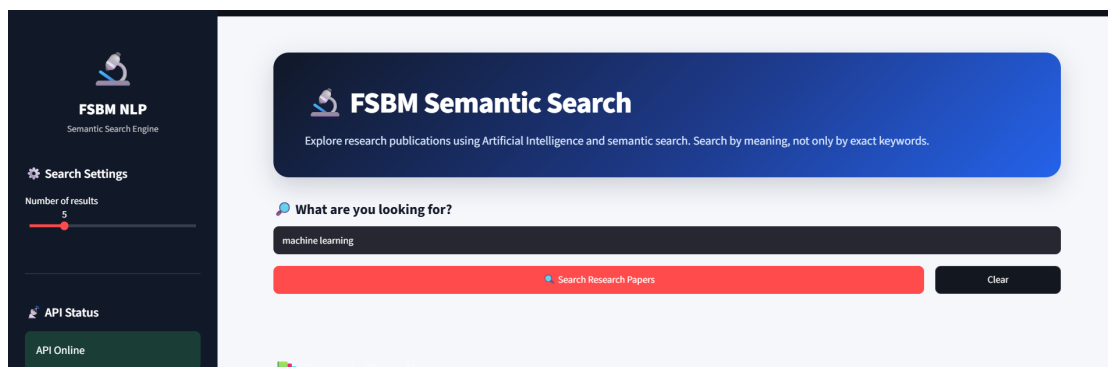


FIGURE 9.1 – Interface Streamlit du moteur de recherche sémantique

9.8.2 Résultats d'une requête

La figure 9.2 illustre les résultats obtenus pour la requête « Diagnostic du cancer du sein par apprentissage automatique », avec les titres, les chercheurs, les années, les citations et les distances sémantiques.

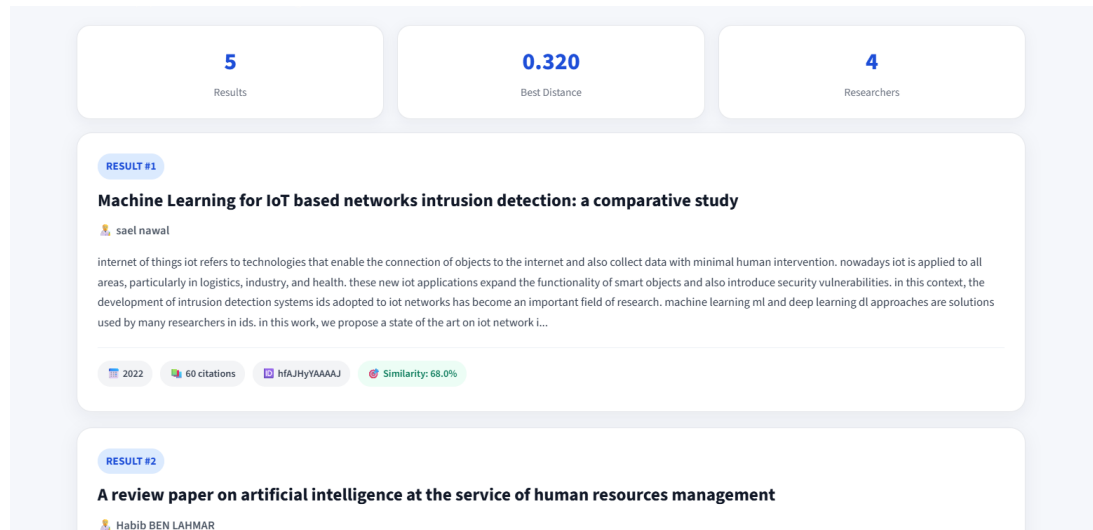


FIGURE 9.2 – Exemple de résultats retournés pour une requête en langage naturel

9.8.3 Projection PCA des embeddings

La figure 9.3 montre la projection en deux dimensions des embeddings de dimension 2560 obtenue par PCA.

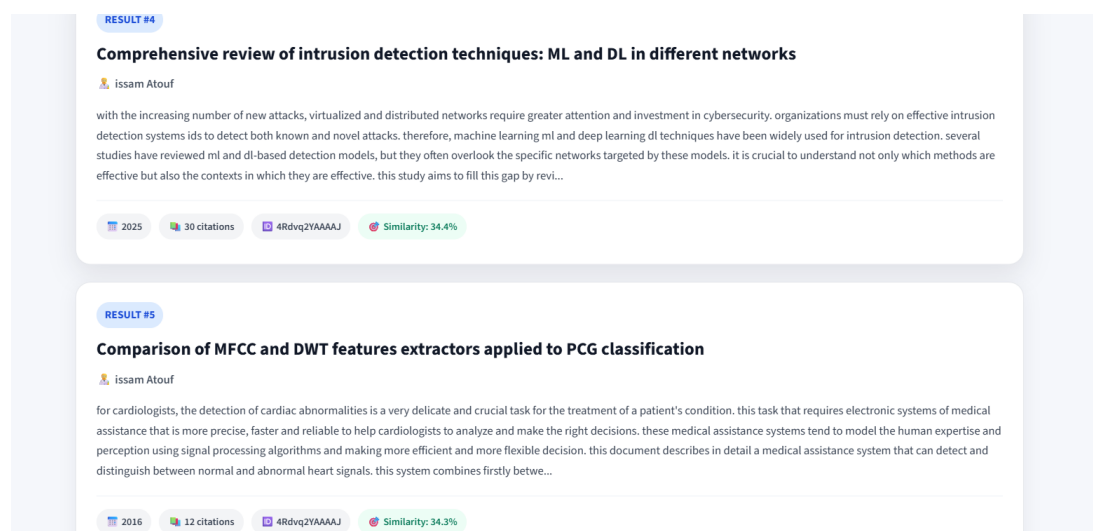


FIGURE 9.3 – Projection PCA des embeddings des publications scientifiques

10. Reproductibilité et dépôt GitHub

10.1 Organisation du dépôt

Le code source est organisé de manière modulaire afin de séparer les différentes responsabilités.

TABLE 10.1 – Organisation des principaux fichiers

Fichier	Responsabilité
<code>scraper.py</code>	Collecte des données scientifiques.
<code>cleaner.py</code>	Nettoyage et structuration des données.
<code>generate_vectors_and_index.py</code>	Génération et indexation des embeddings.
<code>api.py</code>	Exposition du moteur de recherche via FastAPI.
<code>app.py</code>	Interface utilisateur Streamlit.
<code>requirements.txt</code>	Dépendances Python.
<code>semantic_search_demo.ipynb</code>	Démonstration du pipeline.

10.2 Environnement Python

Les principales bibliothèques utilisées sont :

- Python ;
- Pandas ;
- PyArrow ;
- Sentence Transformers ;
- ChromaDB ;
- FastAPI ;
- Uvicorn ;
- Streamlit ;
- Scholarly ;
- Matplotlib ;
- Scikit-learn.

Le fichier `requirements.txt` permet de reproduire l'environnement logiciel.

10.3 Exclusion des fichiers lourds

Les fichiers générés localement ne sont pas nécessairement adaptés à un dépôt Git.

Le répertoire contenant la base vectorielle ChromaDB ainsi que l’environnement virtuel Python sont exclus du dépôt.

Cela permet de conserver un dépôt léger contenant principalement :

- le code source ;
- le dataset final ;
- le notebook ;
- les fichiers de configuration ;
- la documentation.

11. Limites et perspectives

11.1 Limites

Malgré les résultats obtenus, plusieurs limites doivent être prises en considération.

11.1.1 Dépendance à Google Scholar

La collecte dépend d'une source externe dont les mécanismes de protection peuvent évoluer.

Les erreurs réseau, les limitations et les mécanismes anti-bot peuvent affecter la collecte.

11.1.2 Disponibilité des résumés

Toutes les publications ne disposent pas nécessairement d'un résumé accessible ou complet.

Les documents sans contenu textuel exploitable peuvent donc être exclus du processus de vectorisation.

11.1.3 Taille du modèle

Zembed-1 produit des embeddings de dimension 2560 et nécessite des ressources importantes pour son téléchargement et son chargement.

Cela peut représenter une contrainte pour des environnements disposant de ressources limitées.

11.1.4 Évaluation qualitative

Les tests réalisés permettent de démontrer le fonctionnement du moteur, mais une évaluation plus approfondie pourrait utiliser un jeu de requêtes annotées par des experts afin de mesurer des métriques telles que Precision@K, Recall@K ou nDCG.

11.2 Perspectives

Plusieurs améliorations peuvent être envisagées.

1. mettre en place une collecte périodique des nouvelles publications ;
2. ajouter une stratégie de déduplication plus avancée ;
3. améliorer la gestion des erreurs et des reprises ;

4. construire un benchmark de requêtes annotées ;
5. calculer Precision@K et Recall@K ;
6. ajouter des filtres par chercheur, année ou domaine ;
7. ajouter une recherche hybride combinant mots-clés et recherche vectorielle ;
8. améliorer l'interface utilisateur ;
9. déployer l'application dans un environnement serveur ;
10. mettre en place une surveillance des performances du moteur de recherche.

Conclusion générale

Ce projet a permis de concevoir et d'implémenter un pipeline complet de traitement de données scientifiques et de recherche sémantique appliqué à la production scientifique de la Faculté des Sciences Ben M'Sik.

Le pipeline commence par la collecte des profils et publications scientifiques à partir de Google Scholar. Une attention particulière a été accordée à la robustesse de cette étape à travers la gestion des erreurs, la sauvegarde progressive des résultats et la temporisation des requêtes.

Les données collectées sont ensuite nettoyées et structurées. Le prétraitement permet notamment de préserver les caractères accentués français, de traiter les artefacts issus du Web, de détecter la langue des résumés et de produire un dataset exploitable aux formats JSON et Parquet.

La phase de représentation vectorielle constitue le cœur du système. Le modèle `zeroentropy/zembed-1-embedding` est utilisé pour transformer les documents scientifiques en vecteurs de dimension 2560.

Ces représentations sont ensuite indexées dans ChromaDB, ce qui permet d'effectuer des recherches basées sur la proximité vectorielle.

La recherche sémantique est finalement exposée à travers une API FastAPI et une interface Streamlit. L'utilisateur peut ainsi formuler des requêtes en langage naturel et obtenir des publications scientifiques correspondant sémantiquement à sa demande.

Les différentes expérimentations réalisées dans le notebook montrent également l'intérêt de comparer une recherche sémantique à une recherche lexicale classique. La représentation vectorielle permet en effet de rechercher selon le sens des textes plutôt que de dépendre uniquement de la correspondance exacte des mots.

Le projet répond ainsi aux principaux critères définis pour le travail : robustesse de la collecte, qualité du prétraitement, intégration de Zembed-1 et démonstration d'une recherche sémantique fonctionnelle.

Enfin, l'architecture développée constitue une base extensible pour de futures améliorations telles que la recherche hybride, l'évaluation quantitative du moteur, l'actualisation automatique des publications et l'ajout de filtres avancés.

A. Commandes d'exécution

A.1 Création de l'environnement

Sous Windows PowerShell :

```
1 python -m venv .venv
2
3 .\.venv\Scripts\Activate.ps1
4
5 pip install -r requirements.txt
```

A.2 Lancement de l'API

```
1 uvicorn src.api:app --reload
```

L'API est accessible à :

<http://127.0.0.1:8000>

La documentation Swagger est disponible à :

<http://127.0.0.1:8000/docs>

A.3 Lancement de Streamlit

Dans un second terminal :

```
1 streamlit run app.py
```

A.4 Exemple de recherche

Une requête possible est :

```
1 machine learning
```

Une autre requête peut être :

```
1 Diagnostic du cancer du sein par apprentissage automatique
```

Le système encode la requête avec Zembed-1 puis interroge la collection ChromaDB afin de récupérer les publications les plus proches.

B. Extrait de configuration Zembed-1

```
1 from sentence_transformers import SentenceTransformer
2
3 MODEL_NAME = "zeroentropy/zembed-1-embedding"
4
5 model = SentenceTransformer(
6     MODEL_NAME,
7     trust_remote_code=True
8 )
9
10 print(model.get_embedding_dimension())
```

Le résultat attendu pour le modèle utilisé dans ce projet est :

2560

C. Exemple de réponse de l'API

Une réponse du service de recherche est structurée autour de la requête et d'une liste de résultats.

```
1 {  
2     "query": "machine learning",  
3     "results": [  
4         {  
5             "document": "...",  
6             "metadata": {  
7                 "titre": "...",  
8                 "nom_complet": "...",  
9                 "date_publication": "...",  
10                "citations": "..."  
11            },  
12            "distance": 0.32  
13        }  
14    ]  
15 }
```

La distance permet de mesurer la proximité entre la requête et les documents dans l'espace vectoriel.